

Special Number Checks in C

Strong Armstrong Harshad and Duck Numbers

Name Reyansh Pandit

Batch 16

SAP ID 590033602

1 Problem and definitions

The program uses switch-case and four separate parameterless void functions. Each function reads and validates its own positive integer. A Strong number equals the sum of its digit factorials; an Armstrong number equals the sum of each digit raised to the number of digits; a Harshad number is divisible by its digit sum; and a Duck number contains at least one zero.

2 Algorithms

Part	Algorithm
Main menu	Display choices 1 to 5, read the choice, and use switch-case to make a simple call to the selected void function. Repeat until choice 5.
Strong number	Read and validate number. For each digit, calculate factorial iteratively and add it to sum. Compare sum with the original number and display the result.
Armstrong number	Read and validate number. Count its digits. For each digit, multiply it by itself digit-count times and add the power to sum. Compare sum with the original.
Harshad number	Read and validate number. Add all digits. If original number modulo digit sum is zero, it is Harshad; otherwise it is not.
Duck number	Read and validate number. Inspect every digit. If any digit is zero, report Duck; otherwise report not Duck.

3 Test case table

Category	Input	Expected	Observed	Status
Invalid input	choice 1 then -5, 0, 1	Reject -5 and 0; accept 1	As expected	Pass
Strong true	145	Strong	Strong	Pass
Strong false	123	Not Strong	Not Strong	Pass
Strong boundary	1	Strong	Strong	Pass
Armstrong true	153	Armstrong	Armstrong	Pass
Armstrong four digit	9474	Armstrong	Armstrong	Pass
Armstrong false	154	Not Armstrong	Not Armstrong	Pass
Harshad true	18	Harshad	Harshad	Pass
Harshad false	19	Not Harshad	Not Harshad	Pass
Harshad boundary	1	Harshad	Harshad	Pass
Duck true	1023	Duck	Duck	Pass
Duck repeated zero	100	Duck	Duck	Pass
Duck false	123	Not Duck	Not Duck	Pass
Invalid menu	choice 6	Invalid choice	Invalid choice	Pass

4 C program

```
#include <stdio.h>

void check_strong(void);
void check_armstrong(void);
void check_harshad(void);
```

```

void check_duck(void);

void check_strong(void)
{
    int number, original, temp, digit, value, factorial, sum = 0;

    do {
        printf("Enter a positive integer: ");
        scanf("%d", &number);
        if (number <= 0)
            printf("Invalid input. Enter a positive integer.\n");
    } while (number <= 0);

    original = number;
    for (temp = number; temp != 0; temp /= 10) {
        digit = temp % 10;
        factorial = 1;
        for (value = 2; value <= digit; value++)
            factorial *= value;
        sum += factorial;
    }

    if (sum == original)
        printf("%d is a Strong number.\n", original);
    else
        printf("%d is not a Strong number.\n", original);
}

void check_armstrong(void)
{
    int number, original, temp, digit, digits = 0;
    int count, power, sum = 0;

    do {
        printf("Enter a positive integer: ");
        scanf("%d", &number);
        if (number <= 0)
            printf("Invalid input. Enter a positive integer.\n");
    } while (number <= 0);

    original = number;
    for (temp = number; temp != 0; temp /= 10)
        digits++;

    for (temp = number; temp != 0; temp /= 10) {
        digit = temp % 10;
        power = 1;
        for (count = 0; count < digits; count++)
            power *= digit;
        sum += power;
    }

    if (sum == original)
        printf("%d is an Armstrong number.\n", original);
    else
        printf("%d is not an Armstrong number.\n", original);
}

void check_harshad(void)
{
    int number, original, temp, sum = 0;

    do {
        printf("Enter a positive integer: ");
        scanf("%d", &number);
        if (number <= 0)
            printf("Invalid input. Enter a positive integer.\n");
    } while (number <= 0);

    original = number;
    for (temp = number; temp != 0; temp /= 10)
        sum += temp % 10;

    if (original % sum == 0)
        printf("%d is a Harshad number.\n", original);
    else
        printf("%d is not a Harshad number.\n", original);
}

void check_duck(void)
{
    int number, original, temp, has_zero = 0;

```

```

do {
    printf("Enter a positive integer: ");
    scanf("%d", &number);
    if (number <= 0)
        printf("Invalid input. Enter a positive integer.\n");
} while (number <= 0);

original = number;
for (temp = number; temp != 0; temp /= 10)
    if (temp % 10 == 0) {
        has_zero = 1;
        break;
    }

if (has_zero)
    printf("%d is a Duck number.\n", original);
else
    printf("%d is not a Duck number.\n", original);
}

int main(void)
{
    int choice;

    do {
        printf("\n1. Check Strong number\n");
        printf("2. Check Armstrong number\n");
        printf("3. Check Harshad number\n");
        printf("4. Check Duck number\n");
        printf("5. Exit\n");
        printf("Enter your choice: ");
        scanf("%d", &choice);

        switch (choice) {
            case 1: check_strong(); break;
            case 2: check_armstrong(); break;
            case 3: check_harshad(); break;
            case 4: check_duck(); break;
            case 5: printf("Program terminated.\n"); break;
            default: printf("Invalid choice. Select 1 to 5.\n");
        }
    } while (choice != 5);

    return 0;
}

```

5 Compilation

```
gcc -std=c11 -Wall -Wextra -pedantic special_number_checks.c -o special_number_checks
```

Compilation completed successfully with no warnings or errors.

6 Execution

```
(kali[kali])- [~/demo1]
$ gcc -std=c11 -Wall -Wextra -pedantic special_number_checks.c -o special_number_checks
(kali[kali])- [~/demo1]
$ ./special_number_checks

1. Check Strong number
2. Check Armstrong number
3. Check Harshad number
4. Check Duck number
5. Exit
Enter your choice: 1
Enter a positive integer: 145
145 is a Strong number.

Enter your choice: 2
Enter a positive integer: 153
153 is an Armstrong number.

Enter your choice: 3
Enter a positive integer: 18
18 is a Harshad number.

Enter your choice: 4
Enter a positive integer: 1023
1023 is a Duck number.

Enter your choice: 5
Program terminated.
```

Figure 1 Compilation and execution of all four number checks

7 Test result

The compiled program was executed against valid, invalid, boundary, positive, and negative cases for every operation. All observed results matched the prepared expectations.

8 Conclusion

The program meets the switch-case and parameterless void-function requirements and correctly identifies Strong, Armstrong, Harshad, and Duck numbers.